AI CUSTOMER SUPPORT AGENTS

# Decagon, Sierra aur Intercom Fin

Separate deep dives + storefront-to-service flow + custom software blueprint

## Meeting-ready summary

Teeno ka common pattern: customer message -> context + knowledge -> safe decision -> live system action -> answer ya human handoff. Difference control plane mein hai: Decagon AOPs, Sierra Journeys/SDK + model constellation, aur Fin AI Engine + Procedures + documented API.

Is report ka objective: public product behavior ko samajhna, phir ek implementable custom architecture define karna. Vendor ke private databases, services, aur actual network topology ko publicly verified nahi maana gaya hai.

**PUBLIC FACT** Vendor docs mein explicitly disclosed

**INFERRED FLOW** Public features se logical reconstruction

**PROPOSED BUILD** Hamari recommended design, vendor claim nahi

Prepared 05 Oct 2026 | Hinglish technical brief | Sources and confidence labels included

# 01 / Problem aur product map

Real customer-support pain ko architecture controls se connect karo.

| PAIN | ACTUAL FAILURE | SYSTEM CONTROL | MEASURE |
|---|---|---|---|
| **Repeat Q&A** | Answers scattered; agent ka time waste | RAG + current knowledge + citations | Grounded answer rate |
| **Account-specific cases** | FAQ se order / billing state nahi milta | Authenticated tool read + source-of-truth API | Task completion |
| **Risky actions** | Refund/cancel galat ya duplicate ho sakta hai | Policy check + confirmation + idempotency | Unsafe action = 0 |
| **Broken handoff** | User ko sab dobara batana padta hai | Transcript + intent + tool results transfer | Handoff completeness |
| **Stale content** | Purani policy se wrong answer | Versioned ingest + publish/reindex + evals | Freshness lag |

## Product lens

### Decagon

AOP-driven workflow control; chat/voice/email; live integrations, memory, simulations, traces. Strong fit jab operational SOP ko AI execution mein badalna ho. [S1-S3]

### Sierra

Journeys + SDK; composable skills, models by task, supervisors, traces; action aur human handoff. Strong fit complex customer journeys ke liye. [S4-S6]

### Intercom Fin

Documented AI Engine RAG pipeline, Procedures, Intercom workspace aur explicit Fin Agent API. Strong fit existing Intercom stack/integration ke liye. [S7-S9]

# 02 / Decagon - AOP-driven agent

PUBLIC + INFERRED

Public product architecture + explicitly labeled logical runtime reconstruction.

## DOCUMENTED PRODUCT BUILDING BLOCKS

### Agent Operating Procedures

Natural-language SOP ko structured workflow mein convert; LLM flexibility ke saath deterministic validation, tool controls, versioning aur tests. [S1,S2]

### Channels + integrations

Chat, voice, email; helpdesk/CRM/CCaaS, knowledge sync, custom APIs aur MCP; live data se read/action. [S1,S3]

### Operations layer

Simulation, Git-based versions/experiments, traces aur Watchtower-style monitoring se agent behavior inspect/refine. [S1,S2]

## LOGICAL REQUEST PATH - inference from public capabilities, exact private service layout unknown




### Example: return request

1 Customer asks for return -> 2 AOP chooses return procedure -> 3 order API verifies item/date -> 4 policy engine checks eligibility -> 5 action API creates return only after required confirmation -> 6 agent reports case ID or escalates with trace. This is a representative implementation sequence, not Decagon's published wire protocol.

What we cannot verify publicly: exact Decagon HTTP endpoint schema, internal queues/datastores, deployment topology, model routing policy.

Sources: S1-S3 | Full links on last page

# 03 / Sierra - Journeys + model constellation

PUBLIC + INFERRED

Agent Studio/SDK se behavior define hota hai; modular model tasks aur supervisors runtime quality control karte hain.

### Agent Studio / Journeys

Goals, guardrails, step flows, dynamic tools/data, simulations aur trace inspection. CX team behavior design karti hai; engineers integrations/actions jodte hain. [S4]

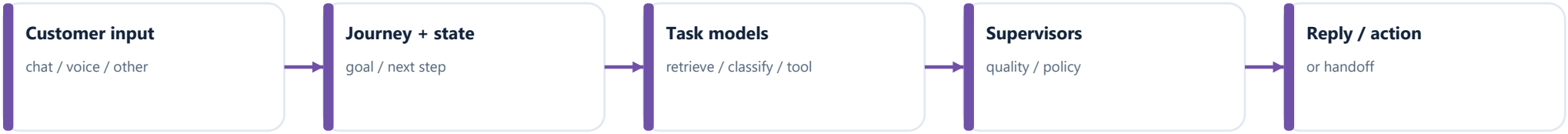
### Agent SDK + skills

Declarative/composable capabilities aur deterministic controls; chat, phone, email, SMS; secure actions, testing, contact-center handoff. [S5]

### Constellation of models

Sierra publicly describes task-specific models for retrieval, classification, tools, policy/tone; supervisors check outcomes; provider redundancy. [S6]

## LOGICAL RUNTIME PATH - reconstructed from public disclosures




### Concrete journey - delayed delivery

User identity -> order lookup -> delivery state -> policy-aware option -> approved tool action (e.g. reschedule) -> supervisor/guardrail check -> response. If tool fails, evidence weak hai, ya policy exception aaye: summary ke saath human handoff. Public sources describe these capabilities; exact network calls and private microservices are not disclosed. [S4-S6]

Implementation takeaway: model ko kaam ke hisaab se select karo, context ko conditional rakho, aur transaction eligibility code/policy se enforce karo.

Sources: S4-S6 | Full links on last page

# 04 / Intercom Fin - documented AI Engine

PUBLIC FACTS

Fin ka publicly described flow: refine -> retrieve / act -> validate -> answer ya escalate.

## FIN'S PUBLISHED LAYERS [S7,S8]

### App layer

Train, test, deploy, analyze; knowledge, guidance/Procedures, audience rules, workflows aur human handoff manage karo.

### AI layer

Retrieval-augmented generation: relevant support content, live data/connectors aur procedure steps choose/augment karta hai.

### Model layer

Retrieval, reranking, summarization, escalation aur response tasks ke liye model components public docs mein described hain.

## AI ENGINE PROCESS [S8]

**1. Refine**
intent, workflow, safety

→

**2. Retrieve / act**
content + data + actions

→

**3. Validate**
relevance + grounding + safety

→

**4. Deliver**
reply or human escalation

### Procedure + handoff

Procedure natural-language instructions ko deterministic controls, data connectors aur actions ke saath combine karti hai. Knowledge sources include Help Center, internal content, PDFs/web pages. Fin clarification maang sakta hai, tool action chala sakta hai, ya user request/negative feedback/policy ke basis par human route kar sakta hai. Output ki citations/grounding aur resolution ko test cases se verify karna zaroori hai. [S7-S9]

Internal service topology, datastore choice aur private model parameters public docs se independently verify nahi kiye ja sakte.

Sources: S7-S9 | Full links on last page

# 05 / Intercom Fin Agent API - actual contract

PUBLIC FACTS

Documented external integration, vendor-internal request processing se alag. Access request + API version 2.14+ required. [S10]

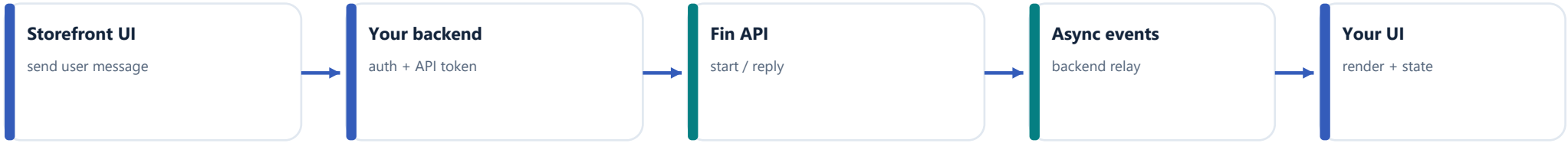
### Mode A - Fin as a tool

Your orchestrator chooses: POST /fin/capabilities (per-user available actions), /fin/ask (one-shot), /fin/procedures/{id}/run, /fin/reply, /fin/escalate. Your app owns routing. [S10]

### Mode B - full turn to Fin

POST /fin/start begins an owned conversation; /fin/reply continues it. Fin handles content, guidance, procedures and escalation, then hands control back on awaiting_user_reply or complete. [S10]

**STORE FRONT / MOBILE APP TO FIN - documented integration pattern**




### Request-response reality

1. Your backend sends bearer token + Intercom-Version; browser never sees secret. 2. Initial HTTP response returns conversation_id, status (often thinking) and optional sse_subscription_url - final answer does not have to be in this response. 3. Backend receives/verifies Fin events, then relays to UI. Fin emits fin_reply_chunk (SSE), fin_replied and fin_status_updated; webhook or SSE can deliver events. 4. Use message.id to dedupe SSE/webhook; final fin_replied replaces cumulative plain-text chunks. 5. Persist final state, render response or handoff. SSE subscription token is short-lived. [S10]

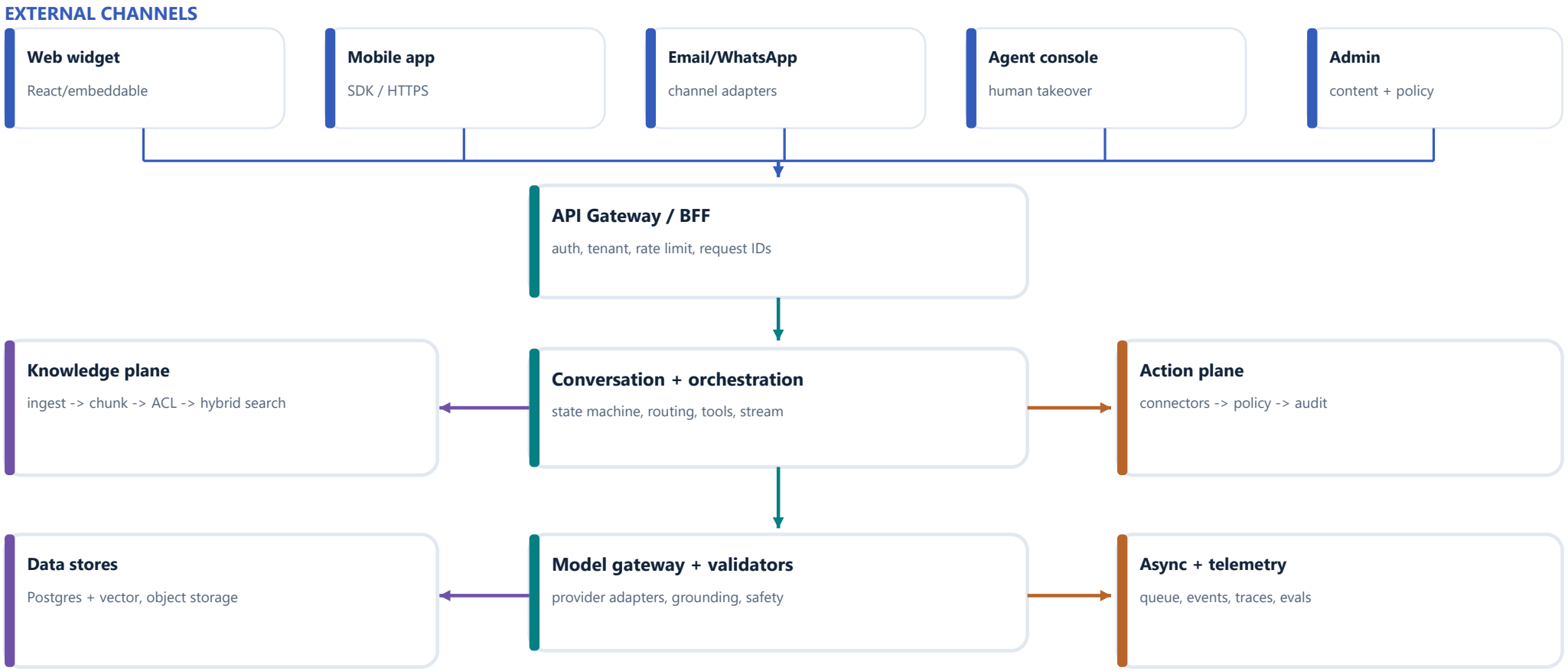
Webhook signature verification: X-Fin-Agent-API-Webhook-Signature. Use documented setup before production. [S10]

Sources: S10 | Full links on last page

# 06 / Custom platform - reference architecture

PROPOSED BUILD

PROPOSED BUILD: deployable design for storefront + admin + agent console; vendor-internal diagram nahi.




### MVP deployment decision

Start modular monolith (conversation, knowledge, action, policy modules) + Postgres/pgvector + object storage + worker/queue. Separate services only when traffic, team ownership ya compliance needs justify karein. Every record carries tenant_id; API and retrieval both enforce tenant scope. Use OpenTelemetry-style traces and provider abstraction from day one.

# 07 / Storefront se final response - exact process

PROPOSED BUILD

PROPOSED BUILD: example user asks 'Order kahan hai? Late ho to shipping fee refund karo.'

| | | |
|---|---|---|
| 01 | **Widget -> API** | UI POST /v1/conversations/{id}/messages; session token + Idempotency-Key; show pending state. |
| 02 | **Gateway + DB** | Verify JWT, tenant/user, rate limit; append message and create turn_id; return 202 + event URL. |
| 03 | **Orchestrator** | Load conversation state; classify 'order status + possible refund'; choose allowed workflow. |
| 04 | **Evidence fetch** | Knowledge search finds current shipping policy; scoped order connector reads live order status. |
| 05 | **Action gate** | If refund eligible: verify identity, compute policy/amount server-side; request explicit confirmation if needed. |
| 06 | **Safe commit** | After confirmation, action service calls payments/order API with idempotency key; audit result + receipt ID. |
| 07 | **Answer stream** | Generate grounded response; validate citations/action result; SSE delta + final; persist final message. |
| 08 | **Fallback** | Low evidence, connector error or policy exception -> queue human, pass summary + timeline + tool results. |

**Non-negotiable: LLM suggests; backend policy decides; system-of-record API commits; UI confirms receipt or human handoff.**

# 08 / Your custom API contract

PROPOSED BUILD

PROPOSED BUILD: example schema; Intercom, Decagon, Sierra endpoint contract nahi.

### POST /v1/conversations/c_123/messages

Authorization: Bearer <user JWT>
Idempotency-Key: user-msg-981

```
{
"text": "Where is order 88?",
"client_message_id": "m_981",
"locale": "en-IN"
}
```

### 202 Accepted

HTTP ack sirf turn creation ka; final answer stream/GET se milega.

```
{
"turn_id": "t_456",
"status": "processing",
"events_url": "/v1/turns/t_456/events"
}
```

### GET /v1/turns/t_456/events (SSE)

#### Event types

`status`, `message.delta`, `message.final`, `action.requires_confirmation`, `handoff`, `error`, done. Each event has event_id, turn_id, sequence.

#### Client + server guarantees

Client reconnects with Last-Event-ID; server dedupes message/action IDs. Final message is stored and available via GET for reload. Error includes stable code + retryable flag. Connector timeout never becomes a fabricated success.

Separate admin APIs: publish knowledge, version policies, run eval suites, inspect traces. Connector credentials remain only on server.

# 09 / Internal data + safety architecture

PROPOSED BUILD

PROPOSED BUILD: actual failure modes ko explicit controls se solve karo.

### Knowledge pipeline

Ingest: Help docs/FAQs/policies -> parse/chunk -> source URL, version, ACL, expiry -> embeddings + keyword index. At query: tenant-filtered hybrid search -> rerank -> relevant snippets -> grounded answer. Publish change triggers reindex + regression eval; stale docs expire.

### Transactional action pipeline

Read: server token -> order/CRM API -> normalized typed result. Write: identity + eligibility + user confirmation + idempotency -> action API -> verified receipt. LLM never chooses amount or calls unrestricted endpoint. Audit input, rule, actor, result.

### Core records

tenants, users, conversations, messages, turns, knowledge_sources, chunks, connector_credentials, tool_calls, action_receipts, handoffs, eval_cases, audit_events. Every query scoped by tenant_id; secrets kept in vault/KMS.

### FAILURE -> CONTROL -> USER EXPERIENCE

| | | |
|---|---|---|
| **Prompt injection in doc** | Treat retrieved text as data; tool allowlist | No unauthorized action |
| **Cross-tenant leak** | Tenant/RLS + ACL filter in search and DB | Only own account content |
| **Tool timeout / stale API** | Bounded retry + circuit breaker + honest status | 'Still checking' or handoff |
| **Duplicate refund/event** | Idempotency key + dedupe + receipt lookup | One charge/refund only |
| **Hallucinated answer** | Citation/claim validation + low-evidence threshold | Clarify or human takeover |

# 10 / Build roadmap - internal to external

PROPOSED BUILD

PROPOSED BUILD: each phase ka deliverable aur release gate clear rakho; dates depend on team and integrations.

| | | | |
|---|---|---|---|
| 0 | **Discovery** | Top 20 intents, policies, source-of-truth APIs, baseline tickets, risk matrix. | GATE<br>**Signed scope + test cases** |
| 1 | **Internal assist** | Knowledge ingest/search, citations, agent console; human approves answers. | GATE<br>**Grounded QA + ACL checks** |
| 2 | **Customer read-only** | Storefront widget, auth, conversation API, streaming, order/account reads. | GATE<br>**Tenant isolation + reliability** |
| 3 | **Controlled actions** | Policy engine, confirmations, action receipts, idempotent connectors, audit. | GATE<br>**Zero unsafe/duplicate actions** |
| 4 | **Human handoff** | Routing, transcript/intent summary, CRM ticket sync, takeover state. | GATE<br>**Full context on handoff** |
| 5 | **Scale + optimize** | Simulations, regression evals, tracing, cost/latency, versioned rollout. | GATE<br>**SLAs + rollback drill** |

Release order: staff first -> limited customer cohort -> broader storefront -> additional channels. Each step needs rollback + versioned policy.

# 11 / Engineering gates + buy/build decision

Before demo, numbers aur traces se proof lo. Vendor choice integration ownership aur workflow depth par depend karega.

## Production acceptance tests

Grounding: answer source cited; no invented order status.
Isolation: tenant A cannot query tenant B docs/orders.
Actions: no refund before policy + verification + confirmation; retry cannot duplicate.
Failure: timeout gives accurate status + handoff.
Handoff: agent sees transcript, intent, evidence, action attempts.
Ops: trace per turn; p95 latency, cost, resolution, escalation measured.

## Decision rule

Fin: strong candidate if Intercom workspace and documented API fit your channels/ownership. Decagon: evaluate if complex SOPs, controlled actions, multi-channel CX dominate. Sierra: evaluate if composable journeys/SDK and model-control approach matter. Custom: justified when UX, data residency, action policy or cross-system ownership require deep control. Validate commercial/security details in vendor due diligence; this report does not infer them.

### Measurement sheet - define baseline before launch

**Resolution**

Verified task completed

**Correctness**

Grounded + policy-safe

**Experience**

CSAT + repeat contact

**Operations**

p95 time + cost/turn

### Meeting takeaway

Client ko demo mein ek real journey dikhao: storefront question -> authenticated live lookup -> policy-grounded decision -> safe action or human handoff -> trace + measurable outcome. Vendor comparison ko same 5-10 golden test cases par run karo, slide claims par nahi.

# 12 / Primary sources + reading notes

All vendor claims in this brief are tied to official vendor documentation/pages. Accessed 05 Oct 2026.

**S1** **Decagon product overview**

https://decagon.ai/product/overview

**S2** **Decagon AOP architecture**

https://decagon.ai/blog/from-sops-to-agent-operating-procedures

**S3** **Decagon integrations**

https://decagon.ai/product/integrations

**S4** **Sierra Agent Studio**

https://sierra.ai/product/agent-studio

**S5** **Sierra Agent SDK**

https://sierra.ai/product/agent-sdk

**S6** **Sierra constellation of models**

https://sierra.ai/blog/constellation-of-models

**S7** **Fin AI Agent explained**

https://www.intercom.com/help/en/articles/7120684-fin-ai-agent-explained

**S8** **The Fin AI Engine**

https://www.intercom.com/help/en/articles/9929230-the-fin-ai-engine

**S9** **Fin Procedures quick start**

https://www.intercom.com/help/en/articles/12599391-quick-start-create-a-fin-procedure

**S10** **Fin Agent API overview**

https://developers.intercom.com/docs/guides/fin-agent-api

Interpretation key: vendor docs show capabilities and selected conceptual pipeline details; diagrams marked inference are architectural reconstruction. Pages 7-11 define a new system recommendation, not a claim about any vendor's private implementation.